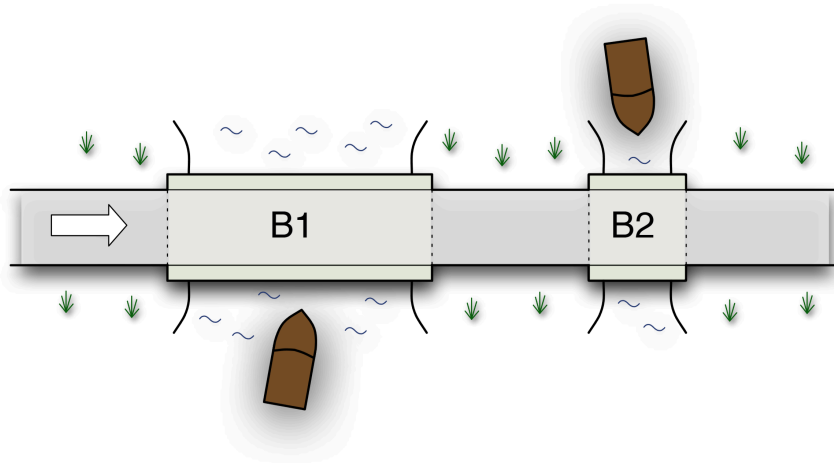


OS_2013-11-08

1 Synchronisation mit Semaphoren (35)

In einem System aus parallelen Prozessen soll die Funktion einer Hebebrückenanlage (siehe Skizze) simuliert werden. Eine Einbahnstraße für Autos kreuzt zwei Schiffahrtswege, wobei die Kreuzungen als Hebebrücken (B1 bzw. B2) ausgeführt sind.



Die unten angegebenen Prozesstemplates sind so mit Semaphoroperationen zu ergänzen, dass die parallele Exekution der Prozesse folgende Eigenschaften erfüllt:

- Im Prozesssystem wird jedes Auto und jedes Schiff durch einen Prozess realisiert. Es gibt drei verschiedene Prozesstemplates: A (für Autos), S1 (für Schiffe, die B1 passieren), und S2 (für Schiffe, die B2 passieren).
- Das System unterstützt beliebig viele aktive Kopien von A, S1 und S2.
- Auf keiner Kreuzung dürfen sich gleichzeitig Autos und Schiffe befinden.
- Auf der Kreuzung B1 dürfen maximal 2 Schiffe gleichzeitig passieren. Auf Kreuzung B2 darf sich zu jedem Zeitpunkt maximal 1 Schiff befinden.
- Die maximale Anzahl von Autos, die sich gleichzeitig auf den Brücken befinden dürfen, ist k für B1 und 1 für B2.
- Der Schiffsverkehr hat Vorrang gegenüber dem Autoverkehr, d.h., Schiffe dürfen nicht durch Autowarteschlangen, die sich vor den Brücken bilden, verzögert werden.
- Autos, die auf das Passieren von B2 warten, halten bereits vor B1, um B1 nicht zu blockieren (d.h., das Straßenstück zwischen B1 und B2 bleibt leer, wenn B2 von einem Schiff passiert wird).
- Das Öffnen und Schließen der Brücken wird zur Vereinfachung durch die Prozesse A, S1 und S2 gesteuert. Der Code dafür ist bereits in den Codefragmenten der Prozesse enthalten und braucht nicht mehr programmiert zu werden.

```
/** Schiff S1 **/
```

```
void S1(void)
```

```
{
```

```
    wait(ship_count)
    wait(ship_mutex)
    ship_on_bridge++

    if (ship_on_bridge == 1) {
        wait(schiff_vor_auto)
        wait(auto_halt)
    }
    signal(ship_mutex)
```

```
    /* Schiff passiert B1 */
```

```
    if (!B1_open) open_B1();
```

```
    passiere_B1();
```

```
    wait(ship_mutex)
    ship_on_bridge--

    if (ship_on_bridge == 0) {
        signal(auto_halt)
        signal(schiff_vor_auto)
    }
```

```
    signal(ship_mutex)
    signal(ship_count)
```

```
}
```

```
/** Auto A **/
```

```
void A(void)
```

```
{
```

```
    wait(ship_b_mutex)
    wait(car_count)
    wait(schiff_vor_auto)

    wait(mutex)
    aufbrücke++

    if (aufbrücke == 1) wait(auto_halt)
    signal(mutex)
    signal(schiff_vor_auto)
```

```
    /* B1 passieren */
```

```
    if (B1_open) close_B1();
```

```
    passiere_B1();
```

```
    wait(mutex)
```

```
/** Schiff S2 **/
```

```
void S2(void)
```

```
{
```

```
    wait (ship_b_mutex)
```

```
    /* Schiff passiert B2 */
```

```
    if (!B2_open) open_B2();
```

```
    passiere_B2();
```

```
    signal(ship_b_mutex)
```

```
}
```

```
/** Semaphorinitialisierungen **/
```

```
    init(car_count, k)
    init(ship_count, 2)
    init(mutex, 1)
    init(auto_halt, 1)
    init(schiff_vor_auto, 1)
    init(schiff_b_mutex, 1)
```

```
aufBrücke--;
```

```
if (aufBrücke == 0) signal(auto_halt)  
signal(mutex)  
signal(car_count)
```

```
/* B2 passieren */
```

```
if (B2_open) close_B2();
```

```
passiere_B2();
```

```
signal(schiff_b_mutex)
```

```
}
```

2 Memory Management – Working Set (20)

Gegeben ist eine Folge von Speicherreferenzen eines Prozesses. Die Entwicklung des *Working Sets* des Prozesses während der Ausführung soll für unterschiedliche *Window Sizes* (Δ) beobachtet werden. Tragen Sie dazu in der unten stehenden Tabelle für jeden Zeitpunkt das Working Set nach der Referenzierung der angegebenen Speicherseite an.

log. Zeit	Referenzierte Seite	Window Size, Δ		
		2	3	4
1	5	5	5	5
2	6	5, 6	5, 6	5, 6
3	7	6, 7	5, 6, 7	5, 6, 7
4	6	6, 7	6, 7	5, 6, 7
5	7	6, 7	6, 7	6, 7
6	8	7, 8	6, 7, 8	6, 7, 8
7	10	8, 10	7, 8, 10	6, 7, 8, 10
8	11	10, 11	8, 10, 11	7, 8, 10, 11
9	13	11, 13	10, 11, 13	8, 10, 11, 13
10	10	10, 13	11, 13, 10	10, 11, 13
11	12	10, 12	13, 10, 12	12, 10, 13, 11
12	13	12, 13	10, 12, 13	13, 12, 10
13	10	10, 13	12, 13, 10	10, 13, 12
14	14	14, 10	13, 10, 14	14, 10, 13, 12
15	14	14	10, 14	14, 10, 13
16	14	14	14	14, 10
17	15	14, 15	14, 15	14, 15
18	14	14, 15	14, 15	14, 15
19	15	14, 15	14, 15	14, 15
20	16	15, 16	14, 15, 16	14, 15, 16

Nehmen Sie an, dass das Lokalitätsverhalten des obigen Prozesses typisch für die Prozesse auf einem Rechner ist. Diskutieren Sie anhand ihrer Tabellenwerte, welchen Wert für Δ Sie bei der Implementierung der Speicherverwaltung für diesen Rechner verwenden würden.

3, weil Thrashing bei den Schleifen vermieden wird und kein 4 wegen Speicher sparen